Exact, hardware-aware local rewriting of quantum circuits, with certified ground truth

Arnav Kapoor[1*]

[1*]Indian Institute of Science Education and Research Bhopal, Bhopal, Madhya Pradesh, India.

Corresponding author(s). E-mail(s): kapoorarnav43@gmail.com;

Abstract

Local rewriting shortens a quantum circuit by repeatedly replacing a short sub-circuit with a cheaper equivalent one taken from a precomputed database. We give this family of methods a precise foundation and test it against exact ground truth. Every useful notion of "equivalent sub-circuit" (exact, up to global phase, on the $|\mathbf{0}\rangle$ input, before measurement, up to a chip symmetry, within ε) is the orbit relation of a group action, so databases are keyed by canonical orbit representatives and soundness and error accumulation follow from two short propositions. On this basis we build a gate-set-agnostic optimiser with exact, state-aware and approximate semantics, calibrated costs and connectivity-aware windows, and extend it with layout-aware rewriting, which places the optimised circuit on the cheapest region of the chip. For the discrete gate sets of a recent proposal every circuit is Clifford. We enumerate all 92,897,280 three-qubit elements and prove when local optimality is global. Beyond Clifford, we use exact arithmetic over $\mathbb{Z}[\mathbf{1}/\sqrt{\mathbf{2}}, e^{i\pi/4}]$ to certify optimal lengths of two-qubit Clifford+T circuits and build long benchmark circuits with planted, certified optima. Against this ground truth the optimiser reaches the proven optimum on 17/20 and 18/20 three-qubit benchmark circuits, where the original method is **4.5**× and **3.0**× above it, and on 41/41 planted Clifford+T circuits. On 58 algorithm circuits compiled by Qiskit for IBM Heron it removes 13% of the gates without adding a two-qubit gate, and layout-aware rewriting raises the estimated success probability over Qiskit's noise-aware layout by ×1.14 (geometric mean). The numerical resynthesis compiler BQSKit needs fewer two-qubit gates than our optimiser on most of these circuits, at the price of approximate outputs, more single-qubit gates and longer run times. The two combine. Run on BQSKit's output, our optimiser removes a further 16% of the gates on Heron, never adds a two-qubit gate and never lowers the estimated success probability. On 22 arithmetic circuits of up to 95 qubits it removes 19% of the gates left by Qiskit's level 3 on Heron. Each of these outputs carries a replay certificate, a log of local steps that

an independent checker verifies a few qubits at a time. At the logical level, on 26 standard circuits in the Nam gate set, it removes more gates than QUESO, Qiskit and TKET and fewer than VOQC, whose global rotation merging it complements. Run after VOQC it gives the best result, 25.3% fewer gates than the input. Every reported output is verified.

Keywords: quantum circuit optimization, peephole optimization, Clifford group, Clifford+T, Cayley graph, exact synthesis, qubit mapping, noise-aware compilation

1 Introduction

Compiling a quantum circuit into a device's native gates leaves redundancy, and on noisy hardware every gate costs fidelity. A common figure of merit, the estimated success probability (ESP), is the product of the fidelities of all gates and measurements [1]. Rosenhahn, Osborne and Hirche [2] recently proposed a simple, gate-set-agnostic remedy. A circuit is viewed as a word over the native alphabet. A window of 2–8 letters is sampled and replaced by a shorter equivalent word, found by random sampling (V1), by lookup in an exhaustively enumerated database of short circuits (V2), or by a lookup gated by a random forest (V3). The method belongs to a long line of database-driven, template and peephole optimisers [3–10].

Two questions decide whether such a method is worth using, and neither is answered in [2]. *How far from optimal is it?* Results there are compared only with Qiskit [11], so it cannot be told whether a 45-gate output for a 3-qubit circuit is good. We show that the optimum is about 10. *Does it help on circuits and hardware that matter?* The gate sets there are discrete $\pm\pi/2$ rotations with all-to-all connectivity, the benchmarks are random words over these alphabets, and the objective is the total gate count.

This paper answers both questions with two strictly separated ingredients, a *general optimiser* that uses no structure of the gate set and *exact ground truth* that does, used only for evaluation.

Contributions.

1. *A unifying model.* Equivalence notions are group actions and database keys are canonical orbit representatives (Table 1). Soundness of state- and measurement-aware rewriting and an additive error bound for approximate rewriting follow (Propositions 1 and 2).
2. *A general optimiser* with exact global-phase handling, calibrated costs and connectivity-aware convex windows (Section 5), and *layout-aware rewriting*: noise-aware relocation of the optimised circuit and layout selection by post-optimisation cost (Section 6).
3. *Theory for discrete gate sets.* An exactness threshold for local rewriting, a parity theorem, mixing of random-circuit benchmarks, and counting lower bounds (Section 7).

4. *Certified ground truth.* Complete distance tables of the 3-qubit Clifford group under both alphabets of [2], and, beyond Clifford, exact two-qubit Clifford+T distances in the ring $\mathbb{D}[\omega]$ together with long benchmark circuits whose optimum is planted and certified (Section 8).
5. *Experiments* on the original benchmarks, on certified Clifford+T benchmarks, and on 58 algorithm circuits for IBM Heron and trapped-ion models and on arithmetic circuits of up to 95 qubits, against Qiskit, TKET [12] and BQSKit [13], and at the logical level against VOQC [14] and QUESO [9], with repeated seeds, an ablation of every component, and every output verified, beyond the reach of statevector simulation by *replay certificates* (Sections 5 and 9).

Section 2 reviews related work, Section 3 fixes notation, Section 10 discusses limitations, and the appendices summarise the reproduction of [2]. The implementation, `qcr`, and every script and raw result are openly available (see Declarations).

2 Related work

Local and database-driven optimisation.

Peephole optimisation with databases of optimal small circuits goes back to reversible logic [3, 4]. For quantum circuits, rule-based optimisers with continuous parameters [5], exact template matching [6] and Clifford templates with symbolic Pauli gates [7] are standard. Superoptimisers enumerate small circuits, collect equivalent pairs and use them as rewrite rules [8, 9], and Quarl learns where to apply them [10]. The method of [2] is the database variant in its simplest form. Our optimiser keeps its simplicity but changes what is stored (canonical orbit keys), where it is applied (DAG-convex windows on coupled qubits) and what it optimises (calibrated cost). Resynthesis compilers take the opposite route. BQSKit [13] partitions a circuit into blocks of a few qubits and synthesises each block anew by numerical optimisation over parametrised circuit templates [15, 16]. It is not limited to stored rewrites, but its blocks are equal to their originals only up to a numerical tolerance.

Optimal synthesis and exact ground truth.

Clifford circuits are described exactly by stabiliser tableaux [17], have a complete set of relations [18], and optimal circuits are known up to six qubits [19], and ZX-calculus simplification gives strong heuristics [20]. Clifford+T unitaries are exactly those with entries in $\mathbb{Z}[1/\sqrt{2}, i]$ [21]. Single-qubit normal forms are T-optimal [22, 23], rotations are approximated optimally [24], and meet-in-the-middle search gives depth- and T-count-optimal circuits for small instances [25, 26]. We use the same ingredients for a different purpose, namely certified optimal lengths over a *native hardware alphabet*, as a benchmark for heuristic optimisers. Benchmarks with known optimal solutions have been advocated for qubit mapping [27], and our planted circuits play this role for gate-count optimisation.

Layout and noise-aware compilation.

Qubit mapping and routing [28] and noise-adaptive mapping [29, 30] choose where a circuit runs, and Qiskit's VF2-based layout passes [11, 31] score placements by

calibrated error rates. Our relocation uses the same subgraph-monomorphism idea but scores placements *after* rewriting and includes measurement errors.

3 Preliminaries

Circuits, devices and costs.

A device fixes n physical qubits, a coupling graph E (or all-to-all), native single-qubit gates, a native two-qubit gate allowed only on pairs in E, and which angles are continuous. A circuit is a word $u = \gamma_1 \cdots \gamma_L$ of native gates that implements $\mathrm{ev}(u) = \gamma_L \cdots \gamma_1$. We use two device families. *IBM Heron* has $\mathrm{RZ}(\theta)$ (virtual), SX, X and CZ on a heavy-hex coupling graph, with per-qubit and per-pair calibrated errors and per-qubit readout errors from Qiskit's calibration snapshots (FakeTorino, 133 qubits, unless stated otherwise). Couplers with calibrated error 1.0 are removed. The *trapped ion* model has all-to-all $\mathrm{RX}, \mathrm{RY}, \mathrm{RZ}(\theta)$ and the Mølmer–Sørensen gate $\mathrm{RXX}(\theta)$ [32] at any angle, with model errors $3 \cdot 10^{-5}$ (single-qubit), 10^{-3} (two-qubit) and virtual RZ. Costs are additive, $c(u) = \sum_i c(\gamma_i)$, with

$$c_{\mathrm{count}} \equiv 1, \qquad c_{w2}(\gamma) = \begin{cases} 10 & \text{two-qubit} \\ 1 & \text{single-qubit} \end{cases}, \qquad c_{\mathrm{cal}}(\gamma) = -\log\big(1 - e(\gamma)\big), \tag{1}$$

where $e(\gamma)$ is the calibrated error of γ on its physical qubits, so $\exp(-c_{\mathrm{cal}}(u))$ is the ESP of u.

Cayley graphs.

For a finite alphabet Γ of gates (modulo global phase) let $G = \langle \Gamma \rangle$. The word length $|g|$ is the length of a shortest word evaluating to g. We write $B(r)$ and $S(r)$ for the ball and sphere of radius r, $D = \max_g |g|$ for the diameter and $\mathrm{Cay}(G, \Gamma)$ for the Cayley graph. The "compute graph" of [2] of depth d is the ball $B(d)$, and its stored "optimal factorisations" are geodesics.

Local rewriting.

With window bound w and database radius r, the rewriting system is $\mathcal{R}(w, r) = \{u \to v : \ |u| \le w, \ \mathrm{ev}(u) \sim \mathrm{ev}(v), \ |v| \le r, \ c(v) < c(u)\}$. Two gates are *independent* if they commute exactly, and words equal up to swapping adjacent independent gates are *trace-equivalent* [33, 34]. A word is (w, r)-reduced if no trace-equivalent word has a factor that is the left side of a rule. The factors of trace-equivalent words are exactly the *convex* subsets of the circuit DAG, so the random commuting shuffles of [2] are a stochastic enumeration of convex windows, which we enumerate directly instead.

4 Equivalence as a group action

A replacement $u \to v$ of a window is *sound* if it does not change what the program computes. What counts as "the same" depends on context, and each notion we need is the orbit relation of a group acting on $U(2^k)$ (Table 1). The database is keyed by a

Table 1 Equivalence notions as group actions. Notions compose. For example, output-aware equivalence modulo chip symmetries uses row-normalised matrices minimised over the automorphism group.

notion	$U \sim V$ iff	group acting	canonical key
exact	$V = U$	trivial	U
up to phase [2]	$V = e^{i\varphi}U$	$U(1)$	phase-normalised U
input-aware	$V\|0\rangle = e^{i\varphi}U\|0\rangle$	stabiliser of $\|0\rangle$	phase-normalised $U\|0\rangle$
output-aware	$V = DU$, D diagonal	diagonal unitaries	rows phase-normalised
symmetry	$V = PUP^\dagger$	coupling-graph automorphisms	minimum over the orbit
approximate	$\|U - V\| \le \varepsilon$	(a tolerance)	exact key, verified distance

hash of a canonical orbit representative, so a single lookup finds every member of the orbit. Exact equivalence including global phase is the only notion that remains sound when the circuit is later controlled. Equality up to phase, used in [2], is kept only for comparison.

Proposition 1 (State- and measurement-aware rewriting) *Let the program be "prepare $|0^n\rangle$, apply $\mathrm{ev}(u)$, measure all qubits in the computational basis", and let S be a convex window of u on wires W. (a) If no gate precedes S on W and $\mathrm{ev}(v)|0_W\rangle = e^{i\varphi}\mathrm{ev}(S)|0_W\rangle$, replacing S by v does not change the output distribution. (b) If no gate follows S on W and $\mathrm{ev}(v) = D\,\mathrm{ev}(S)$ with D diagonal, replacing S by v does not change the output distribution. Any sequence of such rewrites, each applied to the current circuit, preserves the output distribution.*

Proof By convexity, S can be moved to the front (a) or the back (b) of the circuit without changing $\mathrm{ev}(u)$. In case (a) the input factorises as $|0_W\rangle \otimes |0_{\bar{W}}\rangle$, so the final state changes only by the global phase $e^{i\varphi}$. In case (b) the final state changes from $|\psi\rangle$ to $(D \otimes I)|\psi\rangle$, and $|\langle x|(D \otimes I)|\psi\rangle|^2 = |\langle x|\psi\rangle|^2$ for every basis state x. Since the invariant is the output distribution of the current circuit, soundness composes. □

Proposition 2 (Approximate rewriting) *If rewrites $i = 1, \ldots, m$ replace windows S_i by v_i with $\|\mathrm{ev}(v_i) - \mathrm{ev}(S_i)\| \le \varepsilon_i$ in operator norm, the final circuit u' satisfies $\|\mathrm{ev}(u') - \mathrm{ev}(u)\| \le \sum_i \varepsilon_i$ (including global phase in exact mode).*

Proof Telescope over the intermediate circuits $u = u_0, u_1, \ldots, u_m = u'$. Each step changes one factor of a product of unitaries, $\mathrm{ev}(u_i) = A(\mathrm{ev}(v_i))B$ and $\mathrm{ev}(u_{i-1}) = A(\mathrm{ev}(S_i))B$ with unitary A, B, and $\|A(X - Y)B\| = \|X - Y\|$. □

The optimiser reports $\sum_i \varepsilon_i$ with every circuit. Equivalence tests are linear in the operator error (a norm distance), not quadratic as in fidelity tests $|\mathrm{tr}(U^\dagger V)|/2^k \approx 1$, which accept errors of order $\sqrt{\mathrm{tol}}$.

Table 2 Hashed databases of the general optimiser (classes per equivalence notion).

device	topology	notion	depth	classes	build time
Heron	line	up to phase	9	6.31M	6 min
Heron	line	output-aware	10	0.34M	<1 min
Heron	line	input-aware	14	1.41M	<1 min
ion	triangle	up to phase	6	28.4M	38 min
ion	triangle	output-aware	6	7.36M	27 min
ion	triangle	input-aware	7	31.6M	23 min
paper ion ($\pm\frac{\pi}{2}$)	triangle	up to phase	9	2.93M	7 min
paper IBM ($\pm\frac{\pi}{2}$)	triangle	up to phase	10	1.84M	2 min

5 The optimiser

Databases.

For each device we fix a discrete grid of native gates on a three-qubit local topology (all pairs for ions, the line 0–1–2 for heavy-hex). The Heron grid has RZ $\in \{\pm\frac{\pi}{4}, \pm\frac{\pi}{2}, \pi\}$, SX, X and CZ (23 local gates), and the ion grid has RX, RY, RZ on the same angles and RXX($\pm\frac{\pi}{4}, \pm\frac{\pi}{2}$) (57 local gates). Words are enumerated breadth first. For each new class we store a 64-bit hash of its canonical form (Table 1), minimised over the topology's automorphisms, and keep matrices only below the deepest level. Every hit is re-checked numerically before use. Table 2 lists the databases and Fig. 1(b) shows their growth. They are not told that any group is finite or Clifford.

Lookup beyond the radius.

If a window U is not in the database, meet in the middle tries $U = RA$ with A in a stored ball of radius 2, which extends the reach by two gates at the cost of one lookup per element of that ball.

Continuous angles.

Compiled circuits contain arbitrary angles, which no finite grid contains. Windows on at most two *coupled* wires are therefore also resynthesised analytically, on Heron by the KAK decomposition into RZ/SX/X/CZ with the minimal number of CZ [35], on ions by a Weyl-chamber decomposition [36] into one RXX(θ) per non-zero interaction coefficient. Single-qubit runs use Euler decompositions.

Windows.

From each position, a window grows by collecting later gates on a set of at most three wires that commute exactly (tolerance 10^{-10}) past the gates they jump over, so the result is convex. On Heron the wires must induce a connected subgraph, so every replacement is legal without routing. Every prefix of the window is tried, the replacement with the largest cost saving is applied, and sweeps alternate with random exact-commutation shuffles until p consecutive shuffles bring no gain.

Global phase.

Inside windows the optimiser matches up to phase, records each replacement's phase, snapped to the set the device realises exactly ($\pi/4$ multiples on Heron, $\pi/2$ on ions), and repairs the accumulated phase at the end with the cheapest gadget, or by absorbing $\pm i$ into an existing RXX gate on ions, $\mathrm{RXX}(\theta \pm \pi)\,\mathrm{RX}(\pi)^{\otimes 2} = \pm i\,\mathrm{RXX}(\theta)$. The output is equal *including* global phase. The optimiser never returns a circuit of higher cost than its input.

Replay certificates.

Statevector checks end at about 25 qubits, and on routed 23-qubit adders the decision-diagram equivalence checker of [37] returned no result. The optimiser can therefore log every accepted step, namely the swaps of each shuffle and each window replacement. An independent checker replays the log from the input. It accepts a swap if the two gates commute as matrices on their joint wires. It accepts a replacement if every gate it moves past commutes with the window gates it passes, and if the new word equals the old window times a scalar c on the window's wires. Finally it checks that the replayed circuit differs from the output only by the phase repair, that each repaired patch equals its original times a scalar on its own wires, and that the product of all scalars is 1. The checker uses none of the optimiser's databases, canonical forms or commutation rules. Every matrix it builds acts on a few qubits, so its cost grows with the number of steps, not with the circuit width. An accepted certificate proves equality including global phase, up to the floating-point residuals of the local checks, which are summed and reported. Logging does not change the optimiser's output.

6 Layout-aware rewriting

Local rewriting cannot change *where* a circuit runs, yet on calibrated hardware placement matters as much as gate count. On IBM Heron the ESP of the same circuit can differ by an order of magnitude between placements. We add the missing degree of freedom in two ways.

Relocation.

Let u be a routed circuit on physical qubits A, and let H be its interaction graph (the coupled pairs used by its two-qubit gates). A *relocation* is an injective map $\phi : A \to [n]$ such that every edge of H is mapped to an edge of the coupling graph, i.e. a subgraph monomorphism $H \hookrightarrow E$. Relabelling all gates by ϕ gives a native circuit u^{ϕ} whose output distribution is that of u with the measured qubits relabelled, so relocation is exact. Among all relocations we choose one minimising

$$c_{\mathrm{cal}}(u^{\phi}) \;+\; \sum_{q \in A} -\log\bigl(1 - e_{\mathrm{meas}}(\phi(q))\bigr), \tag{2}$$

where the second term accounts for reading out every active qubit. Monomorphisms are enumerated with the VF2 algorithm [31], anchored at every physical qubit (the highest-degree node of H is pinned to it and the search is restricted to the ball of radius

equal to its eccentricity), so the search covers the whole chip while each anchored search stays small. Isolated qubits are placed greedily. Since the identity is a candidate, relocation never increases (2).

Joint layout selection.

The layout that is best before optimisation need not be best after it, because rewriting removes different gates in different placements. We therefore generate candidate layouts (Qiskit level-3 transpilations with several seeds, with and without the calibrated target), optimise each candidate, relocate it, run a final sweep, and keep the candidate of lowest (2).

7 Theory for discrete gate sets

The results of this section concern the alphabets of [2], $\Gamma_{\text{ion}} = \{\text{RX}, \text{RY}, \text{RZ}(\pm\frac{\pi}{2})\} \cup \{\text{RXX}(\pm\frac{\pi}{2})\}$ and $\Gamma_{\text{ibm}} = \{\text{RX}, \text{RZ}(\pm\frac{\pi}{2})\} \cup \{\text{CZ}\}$, modulo global phase.

Lemma 3 (The alphabets generate the Clifford group) *$\langle\Gamma\rangle = \mathcal{C}_n/U(1)$, of order $2^{n^2+2n}\prod_{j=1}^{n}(4^j-1)$. The compute graph of depth d in [2] is the ball $B(d)$ of $\text{Cay}(\mathcal{C}_n/U(1), \Gamma)$.*

Proof All generators map Paulis to Paulis under conjugation, so $\langle\Gamma\rangle \subseteq \mathcal{C}_n/U(1)$. The single-qubit generators generate $\mathcal{C}_1$, and an entangling Clifford together with $\mathcal{C}_1^{\otimes n}$ generates $\mathcal{C}_n$ [18]. Enumeration confirms the orders 24, 11,520 and 92,897,280 for $n = 1, 2, 3$, and all 22 entries of Table I of [2]. □

7.1 When local optimality is global

Theorem 4 (Exactness threshold) *Let G be finite with diameter D. If $w \geq D+1$ and $r \geq D$, every (w, r)-reduced word is a geodesic.*

Proof Let u be (w, r)-reduced. If $|u| > D$, its first $D+1$ letters form a factor f with $|\text{ev}(f)| \leq D \leq r$ and $|f| = D + 1 \leq w$, so the rule $f \to v$ with v a geodesic of $\text{ev}(f)$ applies, a contradiction. Hence $|u| \leq D < w$, so u is itself a factor of length at most w. If it were not geodesic, the rule $u \to v$ with v a geodesic of $\text{ev}(u)$ and $|v| \leq D \leq r$ would apply. □

Proposition 5 (Cascade) *Let $\ell(w)$ be the length of the fixed point of an exhaustive $\mathcal{R}(w, \infty)$ optimiser. If the current length is at most w at any time, the next sweep returns a geodesic.*

Proof If the whole word is a factor of length at most w and the database has unbounded radius, the word itself is replaced by a geodesic of its value. □

The relevant window is therefore the self-consistent $w^\star = \min\{w : \ell(w) \leq w\}$. We measure $w^\star \approx 12$ (ion) and 16 (IBM-style) for 3-qubit circuits, well below $D + 1 = 18$ (Appendix B). For a universal gate set such as Clifford+T the group is infinite, $D = \infty$,

and no finite window is exact. Ground truth must then come from bounded search (Section 8.2).

7.2 Gate-count parity

Theorem 6 (Parity) *(a) For $n \in \{1, 2\}$ and both alphabets, $\mathrm{Cay}(\mathcal{C}_n/U(1), \Gamma)$ is bipartite, hence every equivalent rewrite removes an even number of gates. (b) For $n \geq 3$ and any generating set of $\mathcal{C}_n/U(1)$ that contains, for each qubit, a gate squaring to X_i and one squaring to Z_i, the Cayley graph is not bipartite.*

Proof A Cayley graph is bipartite if and only if there is a homomorphism $\chi : G \to \{\pm 1\}$ with $\chi(\Gamma) = -1$. (a) For $n = 1$, $\mathcal{C}_1/U(1) \cong S_4$ and every $\pm\frac{\pi}{2}$ rotation is a 4-cycle, so $\chi = \mathrm{sgn}$ works. For $n = 2$ we verify bipartiteness exhaustively (no edge joins two elements at equal distance from the identity). (b) The Pauli group $\mathcal{P}_n$ is normal with quotient $\mathrm{Sp}(2n, 2)$. If χ existed, then $\chi(X_i) = \chi(\gamma)^2 = 1$ and likewise $\chi(Z_i) = 1$, so χ would factor through $\mathrm{Sp}(2n, 2)$, which is simple and non-abelian for $n \geq 3$. Hence $\chi \equiv 1$, contradicting $\chi(\Gamma) = -1$. □

Exhaustive search for $n = 3$ gives a shortest odd relation of length $\rho = 11$ (ion) and 9 (IBM-style), so a parity change needs a window of at least $\lceil \rho/2 \rceil = 6$ gates. Fig. 4 of [2], a $6 \to 3$ replacement on two qubits, therefore cannot have been produced with the stated gate set.

7.3 What random-circuit benchmarks measure

Theorem 7 (Benchmarks mix) *Let $\Gamma = \Gamma^{-1}$, let μ_L be the law of $\mathrm{ev}(u)$ for u uniform in Γ^L, and let $\beta < 1$ be the largest modulus of a non-trivial eigenvalue of the random-walk operator (excluding -1 if the graph is bipartite). Then $\|\mu_L - \pi_L\|_{TV} \leq \frac{1}{2}\sqrt{|G|}\,\beta^L$, where π_L is uniform on G (or on the parity class of L). Consequently $\mathbb{E}|\mathrm{ev}(u)| \to \frac{1}{|G|}\sum_d d\,|S(d)|$.*

Proof This is the standard L^2 bound for random walks on groups [38]. By Cauchy–Schwarz, $\|\mu_L - \pi_L\|_{TV} \leq \frac{1}{2}\sqrt{|G|}\,\|\mu_L - \pi_L\|_2$, and the walk operator contracts the component orthogonal to the constants (and, in the bipartite case, to the sign character) by at least β per step. The limit of the expected length follows because $|\mathrm{ev}(u)|$ is bounded by D. □

For $n = 2$ (ion) $\beta = 0.937$. A random length-300 benchmark therefore has a constant optimal answer, 10.72 gates for 3-qubit ion-trap circuits and 12.34 for IBM-style ones, independent of the input length beyond $L \approx 100$.

7.4 Lower bounds for every method

Proposition 8 (Counting bound) *For g uniform in G, $\Pr[|g| \leq d] \leq |B(d)|/|G|$, and $|S(a+b)| \leq |S(a)||S(b)|$. Exact sphere sizes up to depth k therefore give lower bounds on $\mathbb{E}|g|$ that hold for every optimiser.*

Table 3 Exact Cayley-graph data of the Clifford groups generated by the alphabets of [2].

alphabet	n	$\lvert\Gamma\rvert$	$\lvert G\rvert$	diameter	mean $\lvert g\rvert$ (uniform)
ion	2	14	11,520	10	5.89
IBM-style	2	9	11,520	11	6.92
ion	3	24	92,897,280	17	10.72
IBM-style	3	15	92,897,280	19	12.34

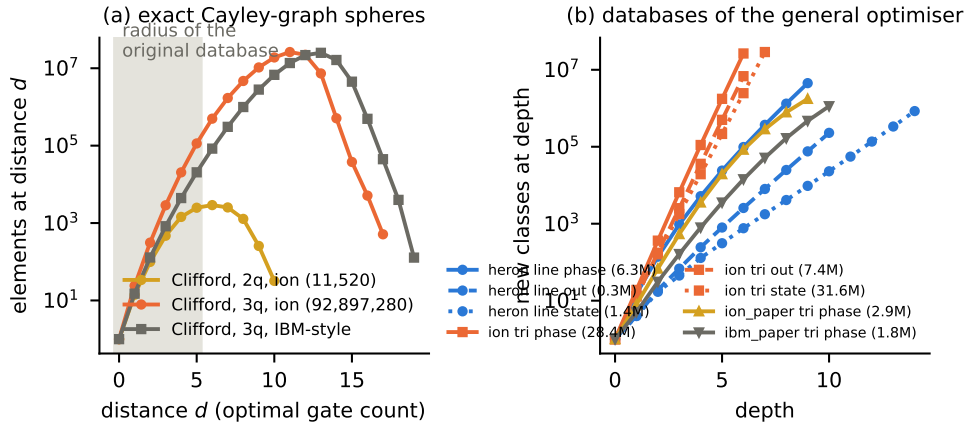


Fig. 1 (a) Sphere sizes of the exact Cayley graphs used as ground truth. The shaded band is the radius of the original database. The Clifford+T sphere sizes keep growing roughly tenfold per level, since the group is infinite. (b) Level sizes of the general optimiser's databases (keys per depth, one curve per device, topology and equivalence notion).

Proof A geodesic of length $a + b$ splits into geodesics of lengths a and b, so $S(a + b) \subseteq S(a)S(b)$. Bounding $|B(d)|$ from above by these products gives $\Pr[|g| \le d]$ from above, and $\mathbb{E}|g| = \sum_{d \ge 0} \Pr[|g| > d]$. □

At $n = 4$ ($|G| = 1.21 \times 10^{13}$), exact spheres to depth 6 give $\mathbb{E}|g| \ge 11.1$ (ion) and ≥ 12.9 (IBM-style).

8 Certified ground truth

8.1 The three-qubit Clifford group

A Clifford modulo phase is determined by its signed tableau [17]. For $n \le 3$ it fits into one 64-bit word, gates act through lookup tables, and a breadth-first level is a vectorised gather. Table 3 and Fig. 1(a) summarise the complete enumeration (25 minutes and about 6 GB per alphabet). Fig. 2 shows a compute graph drawn from our tables: in the Clifford case, distinct short words already collide, so the ball closes up.

For $n = 3$, the numbers of elements that need exactly $0, 1, \ldots, 6$ entangling gates are 13 824, 373 248, 5 971 968, 38 486 016, 42 052 608, 5 971 968 and 27 648 (mean

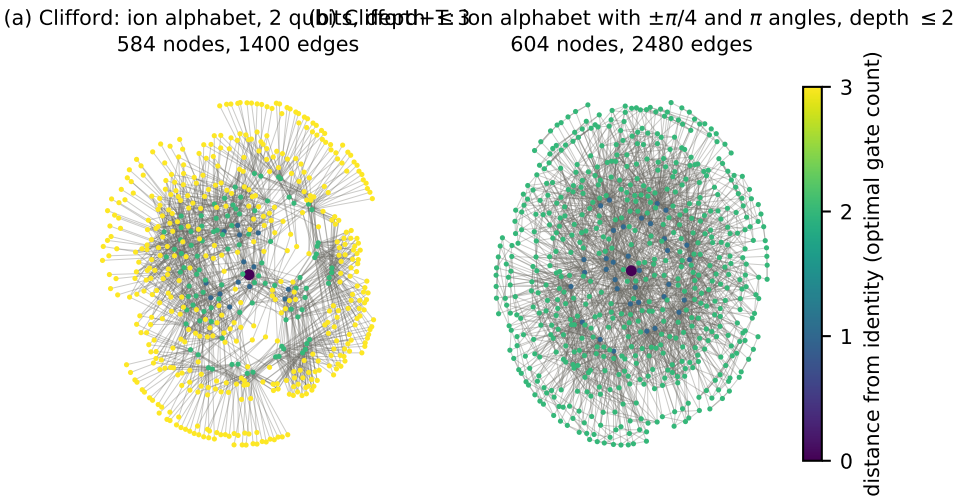


Fig. 2 Compute graphs of comparable size, nodes coloured by exact distance from the identity. (a) Two-qubit Clifford group, ion alphabet, depth ≤ 3. Many short words coincide, and the ball closes up. (b) Two-qubit Clifford+T, depth ≤ 2. Almost every word is new.

3.51). By local equivalence of $\mathrm{RXX}(\frac{\pi}{2})$ and CZ the distribution is the same for both alphabets.

8.2 Beyond Clifford: exact two-qubit Clifford+T distances

With angles in $\{\pm\frac{\pi}{4}, \pm\frac{\pi}{2}, \pi\}$ the ion alphabet ($\mathrm{RX}, \mathrm{RY}, \mathrm{RZ}$ on each qubit and $\mathrm{RXX}(\pm\frac{\pi}{4}, \pm\frac{\pi}{2})$, giving 34 gates on two qubits, the grid of the journal version of [2]) is universal, and the group is infinite. Up to a global phase, however, every generator has entries in the ring $\mathbb{D}[\omega] = \mathbb{Z}[1/\sqrt{2}, \omega]$, $\omega = e^{i\pi/4}$ (e.g. $\mathrm{RZ}(\frac{\pi}{4}) \propto \mathrm{diag}(1, \omega)$, $\mathrm{RX}(\theta) \propto H\,\mathrm{RZ}(\theta)\,H$, $\mathrm{RXX}(\theta) \propto (H \otimes H)\,\mathrm{diag}(1, e^{i\theta}, e^{i\theta}, 1)\,(H \otimes H)$), and so does every circuit [21]. We represent a unitary exactly as $U = M/\sqrt{2}^{\,k}$ with M a matrix over $\mathbb{Z}[\omega]$, stored as four integer coefficients per entry in the $\mathbb{Z}$-basis $1, \omega, \omega^2, \omega^3$.

Lemma 9 (Exact projective canonical form) *For U with entries in $\mathbb{D}[\omega]$ let p be the first non-zero entry in row-major order and $P(U)_{ij} = U_{ij}\overline{U_p}$. Then $P(U)$ has entries in $\mathbb{D}[\omega]$, and $V = e^{i\varphi}U$ for some φ if and only if V has the same first non-zero position and $P(V) = P(U)$. Writing $P(U) = M'/\sqrt{2}^{\,k'}$ with the smallest $k' \geq 0$ for which M' has entries in $\mathbb{Z}[\omega]$ makes the pair (M', k') unique.*

Proof $\mathbb{D}[\omega]$ is closed under products and conjugation ($\overline{\omega} = -\omega^3$). If $V = e^{i\varphi}U$, the zero patterns agree and $V_{ij}\overline{V_p} = U_{ij}\overline{U_p}$. Conversely, $P(V) = P(U)$ at $(i, j) = p$ gives $|V_p| = |U_p|$, so $V_p = e^{i\varphi}U_p$ for some φ, and then $V_{ij} = U_{ij}\overline{U_p}/\overline{V_p} = e^{i\varphi}U_{ij}$. For uniqueness, note that $1, \omega, \omega^2, \omega^3$ is a $\mathbb{Z}$-basis of $\mathbb{Z}[\omega]$ and that the smallest common exponent is unique. Divisibility by $\sqrt{2} = \omega - \omega^3$ is decided exactly by checking that all coefficients of $M\sqrt{2}$ are even. □

We hash the canonical forms into 64-bit keys, grow a breadth-first ball of radius 5 (698,512 elements with level sizes 1, 34, 569, 6,744, 67,664, 623,500, built in 8 minutes) and certify afterwards that the number of distinct keys equals the number of distinct canonical forms, so no collision occurred and the ball is exact. The exact distance of any circuit of length at most $R + R'$ follows by meet in the middle. If $U = XY$ with $|Y| = j \le R'$, then $X = UY^\dagger$ lies in the ball, and $|U| = \min_j\big(j + \min_{Y \in S(j)} |UY^\dagger|\big)$. With spheres kept up to $R' = 5-1$ this certifies optimal lengths up to 9, and a certified lower bound beyond.

Planted benchmarks.

Long circuits are mostly beyond any certifiable radius, so we *plant* the optimum. Starting from a random core c whose exact optimal length equals $|c|$, we repeatedly pick a random window W (up to four gates) and a random word r (up to three gates), compute a geodesic s of Wr^{-1} exactly, and replace W by r followed by s. Since $\mathrm{ev}(s)\mathrm{ev}(r) = \mathrm{ev}(W)$, the unitary, and hence the certified optimum $|c|$, is unchanged. Since windows overlap as the circuit grows, the planted structure is not local. This is the analogue, for gate count, of the known-optimal benchmarks used for qubit mapping [27].

9 Experiments

Verification.

Every output is checked against its input. Exact mode is checked by $\|(U - V)\psi\|$ on random states (statevector on the active qubits, maximum $8.2 \cdot 10^{-11}$), state-aware mode by the total-variation distance (TVD) of the output distributions on $|0^n\rangle$ (maximum $6 \cdot 10^{-12}$), approximate mode against the tracked bound, and Clifford+T outputs by exact equality in $\mathbb{D}[\omega]$. Every output is also checked to be native and to use only coupled pairs. All 250 paper-benchmark outputs, all 720 hardware-suite outputs, and all Clifford+T and layout outputs pass. BQSKit outputs are checked in the same way, up to global phase and BQSKit's final qubit permutation. Outputs of the large-circuit suite (Section 9.5) are checked by replay certificates, and those on at most 24 active qubits also by statevector. Outputs of the logical-level comparison (Section 9.6), of every tool, are checked up to global phase, by statevector up to 20 qubits and by the decision-diagram checker of [37] above.

9.1 The original benchmarks against ground truth

We run the general optimiser with the semantics of [2] (up to phase, gate count, $\pm\frac{\pi}{2}$ alphabets, all-to-all) on the same settings and seeds, namely Tables III/IV (3 qubits, 300 gates, 20 circuits each), the journal's 4-qubit statistics (100 circuits) and Table V (15 qubits, 500 gates, 10 circuits). "Original" is our faithful port of the authors' V2, which reproduces their single-circuit 3-qubit result (47.1 vs. 45 gates).

Table 4 and Fig. 3 show the result. Without knowing that the group is finite or Clifford, the general optimiser is within 0.4 gates of the proven optimum on average and exactly optimal on 35 of 40 three-qubit circuits, where the original method is 4.5×

Table 4 Mean gate counts on the original benchmarks. *Original and Qiskit at 4 qubits use the first 30 circuits. Qiskit is level 3, orig. is the original method, and Cliff. CW uses adaptive commutation-aware windows with the exact Clifford tables (Appendix B), which the general optimiser does not use. Optimum is the proven optimum or a certified lower bound, and #opt. counts the circuits on which the general optimiser is optimal. All 250/250 outputs verified and native.

setting	Qiskit	orig.	Cliff. CW	general	optimum	#opt.
T3, ion, 3q	127.2	47.1	–	10.8	10.4	17/20
T4, IBM-style, 3q	104.0	36.8	–	12.5	12.2	18/20
journal, ion, 4q	164.9*	78.8*	45.6	46.1	≥11.1	–
journal, IBM-style, 4q	140.9*	76.0*	49.3	51.4	≥12.9	–
T5, IBM-style, 15q	332.1	335.9	324.6	327.7	–	–

and 3.0× above the optimum. Fig. 4 shows a typical case. At four qubits it matches the Clifford-specific optimiser, so knowing the group structure is not what makes the difference. Large databases and convex windows are. At 15 qubits all methods, Qiskit included, lie within 4% of one another, because windows see only a small part of a highly entangled random circuit.

9.2 Certified Clifford+T benchmarks

On the two-qubit Clifford+T alphabet we compare the general optimiser (semantics of [2], with an ion database over exactly this alphabet), the original method (compute graph of depth 4 over the same alphabet) and Qiskit level 3 (which emits continuous angles, a different alphabet, and is shown for reference) with the certified optimum.

Fig. 5 summarises 100 random words and 41 planted circuits. On random words with certified optimum the general optimiser is optimal on 99/99 circuits and the original method on 81/99. On planted circuits (mean input 90.3 gates, mean optimum 6.1) the general optimiser reaches the certified optimum on 41/41 (mean 6.1 gates), the original method on 40/41 (mean 6.1), and Qiskit level 3 returns 7.7 gates on average in its continuous alphabet. All outputs of the two discrete methods are exactly equal to their inputs in $\mathbb{D}[\omega]$. Fig. 6 shows an example.

9.3 Algorithms on hardware models

Suite.

We use 60 circuits of 2–12 qubits and up to 1008 gates, taken from Nam/VOQC arithmetic [5], QASMBench [39] (circuits with classical feed-forward and four that do not parse are skipped), QFT, QAOA, Heisenberg Trotter steps, EfficientSU2, a Steane encoder and random circuits. Each is compiled by Qiskit at levels 1 and 3 with the same coupling map and seed. TKET `FullPeepholeOptimise` runs on the level-1 output followed by a rebase. BQSKit [13] (version 1.2.1) compiles the same level-1 output at optimisation levels 2 and 3 with its default settings, restricted to the active qubits. On Heron its model is the coupling graph induced on these qubits, so it may re-place and re-route the circuit within them, and its output is mapped back to the

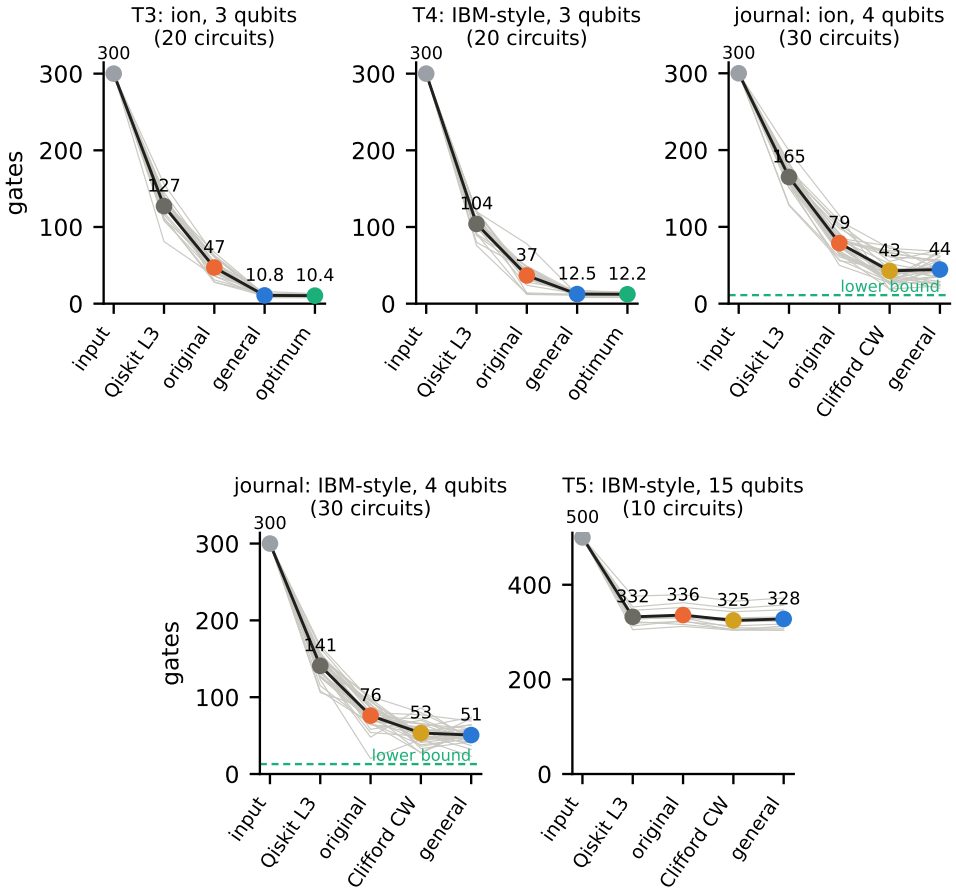


Fig. 3 From the input to the output of every method on the original benchmarks. Grey lines are individual circuits (paired by seed) and the black line is the mean. The dashed line at 4 qubits is the certified lower bound of Proposition 8.

physical qubits so that ESP uses the same calibration. BQSKit resynthesises blocks of up to three qubits numerically to a tolerance of 10^{-8} per block, so its outputs are approximate.

Table 5 and Fig. 7 show the results. *Stacked on Qiskit level 3, the optimiser never adds a two-qubit gate* (9 circuits better and 49 equal on Heron, 1 better and 57 equal on ions) and never lowers ESP. On Heron it removes 13% of all gates. SX/X pulses fall from 7218 to 6139, virtual RZ from 5297 to 4436, and CZ from 2889 to 2830 (Fig. 8). On ions the total can rise (28 circuits) because the calibrated objective trades RX/RY pulses for virtual RZ gates (pulses 3824 → 2422). The guarantee is on the objective and the two-qubit count, not on the total. Starting from level 1 (seed 0, see the robustness check below), the optimiser beats level 3 on Heron in all gates and ESP. Part of the

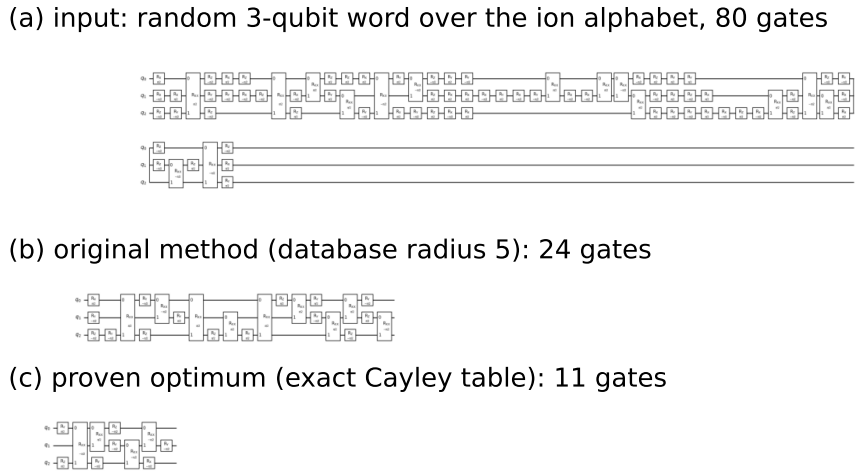


Fig. 4 A random 3-qubit word over the ion alphabet (a), the output of the original method (b) and the proven optimum from the exact Cayley table (c).

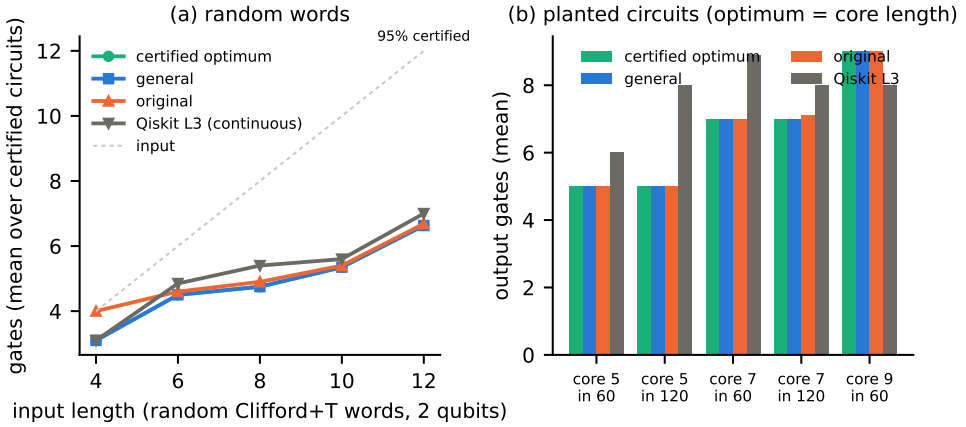
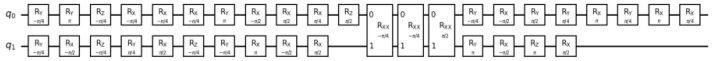


Fig. 5 Clifford+T, two qubits. (a) Random words, mean output length over circuits whose optimum is certified (annotations give the fraction certified). (b) Planted circuits, a core of the stated length hidden in 60 or 120 gates.

ESP gain ($\times 1.11$) is due to Qiskit's level-1 layout already being better than its level-3 layout, our rewriting adding $\times 1.09$. State/measurement-aware rewriting gives the largest reductions, but 5 (Heron) and 7 (ion) circuits are reversible logic on $|0\cdots0\rangle$ with a fixed output and reduce to the empty circuit, so this mode should be read per circuit. Median run time is 10.4 s (Heron) and 3.0 s (ion) per circuit in single-threaded Python.

(a) planted Clifford+T circuit: 36 gates hiding a 5-gate optimum



(b) general optimiser: 5 gates (certified optimum: 5)

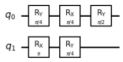


Fig. 6 A planted Clifford+T circuit (a) and the output of the general optimiser (b), which equals the certified optimum.

Table 5 Relative to Qiskit level 3 (58 circuits per device). Count columns are ratios of summed counts, and ESP is the geometric mean of per-circuit ratios. BQSKit outputs are approximate. The last row applies our optimiser to a separate BQSKit level-3 run. Nam pf1_10/pf2_10 are excluded because Qiskit level 3 drops their $\sim 10^{-7}$ rad rotations (approximately, not exactly).

	IBM Heron			trapped ion		
pipeline	2q	all	ESP	2q	all	ESP
TKET (FullPeepholeOptimise on L1)	×1.02	×1.71	×1.16	×1.27	×1.79	×0.992
ours, exact, from L1	×1.00	×0.84	×1.21	×0.95	×0.76	×1.002
ours, exact, from L3	×0.98	×0.87	×1.03	×1.00	×1.00	×1.001
ours, approximate ($\varepsilon = 10^{-3}$), from L1	×1.01	×0.84	×1.21	×0.95	×0.75	×1.002
ours, state/measurement-aware, from L1	×0.77	×0.61	×1.28	×0.62	×0.50	×1.010
BQSKit level 2 on L1 (approximate)	×0.89	×1.26	×1.21	×0.96	×0.74	×1.002
BQSKit level 3 on L1 (approximate)	×0.85	×1.20	×1.22	×0.94	×0.71	×1.002
ours, exact, on BQSKit level 3	×0.85	×1.01	×1.25	×0.94	×0.70	×1.003

Robustness.

All numbers above use transpiler seed 0. We repeated the suite with 5 transpiler seeds for Qiskit and our optimiser (1,624 verified outputs). Stacked on level 3 the result barely moves. Over the seeds the Heron ratios are ×0.975 ± 0.004 in two-qubit gates and ×0.868 ± 0.005 in all gates, with ESP between ×1.03 and ×1.04. Starting from level 1, the all-gate ratio is equally stable (×0.844 ± 0.006), but the ESP gain is not. It ranges from ×0.97 to ×1.21 (mean ×1.10), because it depends on the layout Qiskit picks at level 1. Seed 0 happens to give the largest value. The optimiser's own randomness, the commuting shuffles, changes the total gate count by at most 0.5% over five optimiser seeds on a fixed input.

Comparison with resynthesis.

BQSKit is the strongest baseline in the suite (Table 5). Given the same level-1 input, its level 3 uses fewer two-qubit gates than our exact optimiser on 39 of 58 Heron

circuits and more on 1, and the two reach about the same ESP. Its outputs contain 1.38 times as many gates as ours stacked on level 3, because numerical resynthesis leaves many single-qubit rotations. On ions BQSKit level 3 has fewer gates than every pipeline except our state-aware mode, the same ESP as ours from level 1, and a higher ESP than ours stacked on level 3 on 45 of 58 circuits. Its outputs differ from the input by a median of $1.9 \cdot 10^{-8}$ (maximum $1.4 \cdot 10^{-4}$ on Heron and $1.4 \cdot 10^{-4}$ on ions, measured as $\min_{\varphi} \|(e^{i\varphi}U - V)\psi\|$ on random states), while ours are exact including global phase. BQSKit level 3 takes 98 s per Heron circuit on average with eight worker processes, against 24 s for our single-threaded optimiser.

Exact rewriting on resynthesised circuits.

The two approaches combine. We ran BQSKit level 3 again on the level-1 input and applied our exact optimiser (calibrated cost) to its output. BQSKit is not deterministic across runs, so this input has 2459 two-qubit and 18601 total gates on Heron, close to but not identical with the run in Table 5, and each stacked output is compared with its own input. Our optimiser never adds a two-qubit gate and never lowers ESP. On Heron it removes two-qubit gates on 7 of 58 circuits ($\times 0.996$), reduces the total from 18601 to 15630 gates ($\times 0.84$) and raises ESP on 54 circuits ($\times 1.021$, geometric mean). Most of the saving is in the single-qubit rotations that numerical resynthesis leaves behind. On ions BQSKit's output is already lean, and the gain is small ($\times 0.99$ in total gates, ESP higher on 28 circuits but unchanged in the geometric mean). The total count rises on 4 Heron and 7 ion circuits, where the calibrated cost trades cheap gates for fewer expensive ones. Each rewrite is exact, and the stacked outputs differ from BQSKit's by at most $3.4 \cdot 10^{-10}$ (floating-point rounding). The extra step takes 34 s per Heron circuit and 2 s per ion circuit on average.

9.4 What each component contributes

Table 6 switches off one part of the optimiser at a time on the same 58 circuits per device. On top of Qiskit level 3 on Heron the full optimiser removes 13.0% of all gates. The window length matters most. With $w = 8$ the saving falls to 7.9% and with $w = 4$ to 1.3%. The database comes next. Without it the saving is 5.8%, and restricting it to words of at most three gates gives 6.2%. This is the hardware counterpart of Section 8, where window length and database radius decide how close local rewriting gets to the optimum. Analytic resynthesis contributes most of the rest on Heron (8.2% without it) and is what reduces the Mølmer–Sørensen count on ions. Without it, starting from level 1, the ion circuits need $\times 1.29$ the two-qubit gates of level 3 instead of $\times 0.95$. Convex windows and the commuting shuffles add a little each, and meet in the middle almost nothing on this suite (12.3%). The calibrated cost gives the highest ESP. Counting gates instead removes a similar number of gates but raises ESP only by $\times 1.010$ instead of $\times 1.027$.

Table 6 Ablation on the hardware suite (58 circuits per device, seed 0), relative to Qiskit level 3. Each row changes one component of the full optimiser. Count columns are ratios of summed counts, ESP is the geometric mean of per-circuit ratios. All outputs verified.

	Heron, from L3			Heron, from L1		ion, from L1	
variant	saved	2q	ESP	all	2q	all	2q
full optimiser	13.0%	×0.98	×1.03	×0.84	×1.00	×0.76	×0.95
contiguous windows	11.7%	×0.99	×1.03	×0.88	×1.03	×0.82	×0.99
window $w = 8$	7.9%	×0.99	×1.02	×0.96	×1.17	×0.81	×0.97
window $w = 4$	1.3%	×1.00	×1.00	×1.09	×1.27	×1.06	×1.19
no shuffles	10.3%	×0.98	×1.03	×0.87	×1.02	×0.81	×0.98
no database	5.8%	×0.99	×1.02	×0.95	×1.04	×1.08	×1.01
database radius ≤ 3	6.2%	×0.99	×1.02	×0.94	×1.04	×0.89	×0.99
no meet in the middle	12.3%	×0.98	×1.03	×0.86	×1.02	×0.76	×0.95
no resynthesis	8.2%	×0.98	×1.02	×0.97	×1.15	×0.87	×1.29
gate-count cost	11.6%	×1.00	×1.01	×0.87	×1.11	×0.73	×1.08
2q-weighted cost	11.7%	×0.98	×1.02	×0.84	×1.01	×0.74	×0.95

Table 7 Large circuits, relative to Qiskit level 3 (22 Heron and 25 ion circuits of up to 95 qubits). Count columns are ratios of summed counts, and ESP is the geometric mean of per-circuit ratios over the 13 Heron and 25 ion circuits whose Qiskit level-3 ESP is at least 10^{-3}. Below that, ESP is dominated by noise and a ratio of two such numbers swings by orders of magnitude. Every output of our optimiser has an accepted replay certificate.

	IBM Heron			trapped ion		
pipeline	2q	all	ESP	2q	all	ESP
Qiskit level 1	×1.22	×1.13	×0.24	×1.40	×1.13	×0.851
TKET (FullPeepholeOptimise on L1)	×1.07	×1.96	×0.27	×1.39	×1.87	×0.828
ours, exact, from L1	×1.05	×0.78	×0.43	×0.96	×0.68	×1.033
ours, exact, from L3	×0.98	×0.81	×1.12	×1.00	×1.00	×1.011

9.5 Large circuits

Suite.

To test scale we ran the same pipelines on 45 circuits of the Nam/VOQC suite [5] with 14 to 100 qubits. They are decomposed to $\{U3, CX\}$ and compiled by Qiskit for the 133-qubit Heron model and for an all-to-all ion model with one ion per qubit. Our optimiser runs in the calibrated exact mode as before. The product-formula circuits pf1 and pf2 (19 per device) are excluded from the comparison for the reason given for Table 5, as Qiskit level 3 removes almost all of their gates. Circuits with a certificate awaiting re-check (2 on Heron, 1 on ions, see below) are excluded as well. This leaves 22 Heron circuits of 14–95 qubits with up to 5,425 input gates, and 25 ion circuits with up to 5,425 input gates. BQSKit levels 2 and 3 ran on the 18 smallest circuits of the suite before the run was stopped for time.

Results.

For gate counts, Table 7 shows the picture of the 60-circuit suite with larger margins on Heron. Stacked on Qiskit level 3, the optimiser removes 19% of all gates and 2.1% of the two-qubit gates on Heron. It removes two-qubit gates on 20 of 22 circuits, never adds one, and raises ESP by $\times 1.12$ in the geometric mean over the circuits of Table 7 (median $\times 1.14$ over all 22). On ions it never adds a two-qubit gate and never lowers ESP. The total never rises on Heron and rises on 12 ion circuits, again because the calibrated cost trades pulses for virtual RZ gates. Starting from level 1, the optimiser ends with fewer gates than level 3 on all 22 Heron circuits but with more two-qubit gates on 16 of them, since level 3 routing and resynthesis find entangling structure that local rewriting does not reach. Its ESP from level 1 is also lower, because at this scale the level-3 layout and routing matter more than the gates rewriting removes. On ions, where no routing is needed, it uses fewer two-qubit gates than level 3 on 11 circuits and more on none.

Resynthesis at scale.

On the 14 comparable circuits of up to 36 qubits that BQSKit completed, the picture of the 60-circuit suite holds. Relative to Qiskit level 3 on Heron, BQSKit level 3 reaches $\times 0.91$ two-qubit gates, fewer than ours stacked on level 3 ($\times 0.98$), but $\times 1.35$ in all gates, against $\times 0.82$ for ours. On ions BQSKit level 3 reaches $\times 0.95$ and $\times 0.63$, and ours from level 1 $\times 0.96$ and $\times 0.53$. BQSKit's outputs on at most 24 qubits differ from their inputs by up to $3 \cdot 10^{-4}$. Its 22 outputs on more qubits could not be checked, as the decision-diagram checker returned no result.

Certificates.

Of the 176 outputs of our optimiser, 171 have an accepted replay certificate. A certificate replays up to 544,922 steps in at most 6.8 s (median 0.13 s), against up to hours for the optimisation itself, and the summed residuals stay below 10^{-7}. The first version of the checker rejected 7 certificates. It checked the whole phase repair as one block, which exceeds its limit of 12 wires when the repair touches two distant parts of a wide circuit. The current checker treats each repaired patch separately. The optimiser is deterministic. Rerunning it on the 2 cases re-checked so far reproduced their outputs exactly, and the current checker accepted both. The remaining 5 were not re-checked and are excluded from Table 7.

Run time.

Table 8 lists median run times. Qiskit needs well under a second per circuit and TKET a few seconds. On the 60-circuit suite our optimiser takes seconds per circuit, like BQSKit level 2 and less than BQSKit level 3, which used eight worker processes. On the large suite it takes minutes and up to 6.4 h per circuit, in single-threaded Python with several circuits sharing eight cores. BQSKit level 3 needs a median of several minutes per Heron circuit on the part of the suite it completed. Run time is the main cost of the method at this scale. Sweeps over non-overlapping windows could run in parallel, which we have not done.

Table 8 Median run time per circuit in seconds. Qiskit level 1 was not timed on the 60-circuit suite. On the large suite BQSKit covers only the 18 smallest circuits, so its medians are over easier circuits than the others, and part of its run shared the machine with an unrelated job. The large suite ran several circuits at a time on eight cores.

	60-circuit suite		large suite	
pipeline	Heron	ion	Heron	ion
Qiskit level 1	–	–	0.11	0.04
Qiskit level 3	0.03	0.01	0.12	0.05
TKET	0.30	0.86	8.2	9.6
BQSKit level 2 (8 workers)	14.7	4.8	132	34.2
BQSKit level 3 (8 workers)	53.6	11.2	842	138
ours, exact, from L1	10.4	3.0	853	187
ours, exact, from L3	6.0	4.8	117	57.1

Table 9 Logical level, Nam gate set, summed over 26 circuits. Changes are relative to the input. The last column counts the circuits on which a tool is best or tied among the single tools. All outputs verified.

tool	gates	change	CNOT	change	best on
input	10,481	–	4,612	–	–
Qiskit level 3	10,391	-0.9%	4,511	-2.2%	0
TKET (FullPeepholeOptimise)	11,762	12.2%	4,489	-2.7%	0
QUESO (5 min per circuit)	9,765	-6.8%	4,465	-3.2%	4
VOQC	8,417	-19.7%	4,314	-6.5%	23
ours (exact up to phase, gate count)	9,480	-9.6%	4,426	-4.0%	2
VOQC, then ours	7,833	-25.3%	4,007	-13.1%	–

9.6 Logical-level comparison

Quartz [8], QUESO [9] and VOQC [14] are evaluated in a different setting, the Nam gate set $\{H, X, \mathrm{RZ}, \mathrm{CNOT}\}$ with all-to-all connectivity and total gate count as the objective, on 26 arithmetic and Toffoli circuits of [5]. Our optimiser runs there unchanged, with a database over this gate set, equality up to global phase and the gate count as cost. Every tool starts from the same input files, the decompositions distributed with QUESO. VOQC runs its full Nam pass sequence and QUESO its recommended rule sets with a budget of five minutes per circuit (the QUESO paper uses one hour, which we could afford only for 3 small circuits, where it improved the total from 305 to 282 gates). All 159 outputs are equal to their inputs up to global phase.

Table 9 shows a clear order. VOQC removes 19.7% of the gates and is best on most circuits. Our optimiser removes 9.6% and is second, ahead of QUESO with five minutes, Qiskit and TKET. VOQC's advantage comes from rotation merging over phase polynomials, which combines RZ gates arbitrarily far apart, while our windows

Table 10 Mean noisy-simulation TVD by objective (lower is better).

	Qiskit L1	Qiskit L3	ours, count	ours, $w2$	ours, calibrated
Heron	0.115	0.137	0.096	0.088	0.088
ion	0.0157	0.0091	0.0092	0.0084	0.0084

Table 11 Layout-aware rewriting on IBM Heron. ESP includes readout, and ratios are geometric means relative to Qiskit's noise-aware level 3. TVD: mean exact noisy TVD including readout (5 circuits).

pipeline	ESP ratio	two-qubit gates (sum)	TVD
Qiskit L3 (coupling map only)	×0.63	493	0.569
Qiskit L3, noise-aware	×1.00	493	0.332
noise-aware + `qcr`	×1.01	485	0.325
noise-aware + `qcr` + relocation	×1.14	485	0.225
joint layout selection (`qcr`)	×1.14	471	0.222

see three qubits at a time. Our windows in turn find rewrites VOQC does not. On `mod5_4` ours reaches 35 gates against 56 for VOQC. As with resynthesis, the two combine. Our optimiser applied to VOQC's output improves it on 21 of 26 circuits and gives the best result overall, 25.3% fewer gates and 13.1% fewer CNOTs than the input. Quartz was not run.

9.7 Choosing the objective

We ran the optimiser with c_{count}, c_{w2} and c_{cal} and simulated each output with noise (on Heron the calibrated gate errors of the actual physical qubits, on ions depolarising noise at the model rates), using the exact TVD between ideal and noisy output distributions from density matrices on the 49 circuits with at most eight active qubits (Table 10). The calibrated cost is at least as good as the alternatives on both devices and beats Qiskit level 3 on 39/49 Heron circuits. The two-qubit-weighted count is a good proxy where no calibration exists.

9.8 Layout-aware rewriting

On Heron we compare, for the 8 circuits of the suite, Qiskit level 3 without and with the calibrated target (noise-aware layout), our optimiser on each, relocation after the noise-aware pipeline, and joint layout selection over four seeds × two modes. ESP here includes readout errors of the active qubits, and the noisy simulation now includes readout errors as well.

Table 11 and Fig. 9 show that placement dominates: Qiskit's own noise-aware layout already raises ESP by a factor of 1/0.63 over the coupling-map-only layout. Relocation after rewriting adds ×1.14, and joint selection ×1.14, with noisy TVD lower than the noise-aware baseline on 5/5 circuits. Joint selection chose a noise-aware

candidate in 50% of the circuits, i.e. the best final layout is often not the one Qiskit ranks best before optimisation.

10 Discussion and limitations

What the ground truth shows.

For the discrete alphabets of [2], local rewriting is rewriting on a Cayley graph, controlled by a few exact quantities (diameter, growth, shortest odd relation, mixing rate), and the original method leaves most of the possible reduction unrealised because its database radius is below the exactness threshold. A general optimiser with large hashed databases and convex windows closes almost all of the gap without exploiting the group structure, also on certified Clifford+T circuits.

Exact rewriting and resynthesis.

The BQSKit comparison separates two kinds of optimisation. Resynthesis finds cheaper entangling structure inside blocks that no stored rewrite reaches, and is therefore better at reducing two-qubit gates. Exact local rewriting is faster, keeps the unitary including global phase, and removes more single-qubit gates. The two are complementary rather than competing. Applied after resynthesis, exact rewriting keeps BQSKit's two-qubit savings and removes a further 16% of the gates on Heron at no cost in accuracy. The same holds for VOQC's global rotation merging at the logical level, where exact local rewriting on top gives the best result of all tools. A practical pipeline would use a global or resynthesis pass for structure that windows cannot see and exact local rewriting to clean up.

Limitations.

(i) *Noise models, not hardware.* ESP and TVD come from calibration snapshots. The simulations contain gate, relaxation and readout errors but no idle errors or crosstalk, and no real-hardware runs were performed. (ii) *Global phase.* Our outputs are exact including phase, which costs 240 single-qubit gates over the Heron suite. With phase as metadata, as Qiskit, TKET and BQSKit store it, the all-gate ratio from level 1 improves from $\times 0.838$ to $\times 0.825$. (iii) *Baselines.* Quartz was not run, and QUESO had five minutes per circuit instead of one hour. Qiskit, TKET and BQSKit use their default settings, BQSKit level 4 was run only on the four-qubit benchmark (Table B3), and on the large suite BQSKit covers only the 18 smallest circuits. BQSKit is not deterministic, so its numbers carry a run-to-run spread that we did not measure beyond the two runs of Section 9. (iv) *Scale.* Windows have at most three qubits, the code is single-threaded Python, and feed-forward circuits are out of scope. Circuits of up to 95 qubits and 5,425 gates were tested, where one run takes up to a few hours. Two 64-qubit Heron runs (QFT64 and QFTAdd32) did not finish, and the 5 outputs whose certificates were not re-checked are left out of Table 7. (v) *Clifford+T ground truth* is certified for two qubits and lengths up to 9, and three-qubit Clifford+T balls grow too fast for the present implementation. (vi) *Verification* is numerical (random input states) for the 60-circuit hardware suite and by replay certificates for the large suite.

A certificate proves equality up to the rounding of its local checks. Exact arithmetic is used only for Clifford and Clifford+T circuits.

Future work.

Two questions remain open. The first is a theory of the self-consistent window $w^\star$, relating the cascade to Dehn presentations and local geodesics in groups [40, 41] and to complete rewriting systems [42]. The second is how to reach the entangling structure that resynthesis finds without giving up exactness, through larger windows with four-qubit databases and symmetry reduction, or through resynthesised blocks whose angles are snapped to exactly realisable values and then verified. Error-barred runs on real hardware are the natural next experimental step.

11 Conclusion

Local rewriting becomes reliable when "equivalent" is made precise. Each notion is a group action, the database stores its orbits, and soundness and error accumulation follow. A general optimiser built this way, without using the structure of any gate set, reaches proven optima on the original three-qubit benchmarks and on planted Clifford+T circuits, improves Qiskit's level-3 output on real algorithms for Heron and ion models without adding two-qubit gates, and, combined with layout-aware rewriting, raises the estimated success probability beyond Qiskit's noise-aware layout. Applied after numerical resynthesis (BQSKit), it keeps the resynthesised two-qubit structure and removes a further 16% of the gates on Heron. Exact ground truth, Clifford and beyond, is what makes such claims checkable.

Acknowledgements. We thank the authors of [2] for publishing their demonstration code, and the developers of Qiskit, Qiskit Aer, TKET, QASMBench and the VOQC benchmarks.

Declarations

- **Funding:** No funding was received for this work.
- **Competing interests:** The author declares no competing interests.
- **Ethics approval and consent to participate:** Not applicable.
- **Consent for publication:** Not applicable.
- **Data availability:** All raw results, logs and figure data are included in the repository below (`results/`, `figures/`).
- **Materials availability:** Not applicable.
- **Code availability:** The implementation (`qcr`, Apache License 2.0), every experiment script and the scripts that generate all tables, figures and numbers in this paper are available at https://github.com/arnavk23/qcr.
- **Author contribution:** A.K. conceived the study, developed the software, performed the experiments and wrote the manuscript.

Appendix A Reproduction of the reference method

We ported the authors' MATLAB demonstration code line by line to Python and reran every table and figure of [2]. Table A1 summarises the outcome (details in the reproduction report in the repository).

Table A1 Reproduction of [2].

claim	status	detail
Table I (compute graphs)	exact	identical node and edge counts
Figs. 1, 4, 5 (examples)	reproduced	Fig. 4 requires a different gate pool (Theorem 6)
Tables III/IV, journal 4q	qualified	only with the 24-operator, depth-5 database, which is not stated
Table II (V1 > V2 > V3)	qualified	V3 wins only with a tuned forest threshold (recall 30% at the default)
Table V (15 qubits)	not reproduced	ties a current Qiskit (335.9 vs. 332.1 gates)
Fig. 9 (hardware)	qualified	present at Qiskit levels 0/1, absent at level 3

Appendix B Clifford-specific optimisers

For the alphabets of [2], the exact tables give an *exact-window* optimiser EW-w (all windows of length at most w on at most three wires replaced by optimal words) and its DAG-convex version CW-w. CW-sem grows windows adaptively while gates commute into them.

Table B2 Mean gate counts in the setting of Tables III/IV of [2] (3 qubits, 300 gates, 20 circuits).

	optimum	original	EW-8	EW-16	Qiskit L3	paper (1 circuit)
ion	10.4	47.1	28.3	10.4	127.2	45
IBM-style	12.2	36.8	51.1	12.2	104.0	72
time per circuit (ion)	–	51 s	0.54 s	0.24 s	<0.1 s	–

Findings.

(1) On its own benchmark the original method finishes 4.5× (ion trap) and 3.0× (IBM-style) above the exact optimum. Our faithful port reproduces the paper's single-circuit number (47.1 vs. 45 gates), so the gap is a property of the method, not of our implementation. (2) EW-w reaches the *proven* optimum on all 20 circuits once $w \geq 12$ (ion trap) or $w \geq 16$ (IBM-style), in a fraction of a second. This is the cascade of Prop. 5. The transition is sharp, between $w = 8$ and $w = 12$ (ion trap) or $w = 16$

(IBM-style). (3) The grid (Fig. B1) shows that the *database radius*, not the window or the search strategy, is the binding constraint. At $r = 5$, enlarging the window from 8 to 16 lowers the mean excess only from 46 to 43 gates. At $w = 8$, raising r from 5 to ∞ lowers it from 46 to 18, and for $w \geq 12$ with $r \geq 8$ the excess is at most 2 gates (ion trap, and from 72 to ≤ 8 for the IBM-style set). Theorem 4 predicts both requirements, windows long enough *and* a database deep enough. The original method is limited by the second.

Table B3 Journal benchmark (4 qubits, 300 gates, 100 circuits). Original (orig.) and Qiskit level 3 use the first 30 circuits. Published (publ.) numbers are from the journal version of [2]. BQSKit level 4 is our run on the first 30 circuits, with the same gate set at continuous angles (approximate, all-to-all).

	bound	CW-sem	CW-24	EW-16	orig.	publ.	Qiskit	BQSKit
ion, total	$\geq$11.1	45.6	56.7	110.3	78.8	111	164.9	120.6
ion, RXX	–	–	27.6	44.2	33.3	43	47.3	27.6
IBM-style, total	$\geq$12.9	49.3	59.1	130.9	76.0	107	140.9	165.2
IBM-style, CZ	–	–	27.1	47.1	32.4	43	47.3	35.1

Findings.

(1) *Convexity matters more than anything else at $n \geq 4$.* With the same exact 3-qubit oracle, contiguous windows (EW-16) give 110.3 (ion trap) and 130.9 (IBM-style) gates, no better than the published method. With DAG-convex windows (CW-24) the same oracle gives 56.7 and 59.1 gates, i.e. 49% and 45% fewer than the published results, in about 5.6 s per circuit. (2) Two-qubit counts drop from 43 (published) to 27.6 RXX and 27.1 CZ, although CW optimises total length, not two-qubit count. On the first 30 circuits, where we also ran BQSKit level 4 (Table B3), CW-24 averages 27.0 RXX against BQSKit's 27.6 (fewer on 16, more on 13) and 28.1 CZ against 35.1 (fewer on 25, more on 4), with $\times 0.46$ and $\times 0.37$ of BQSKit's total gates and exact outputs. The journal reports 28 and 37 two-qubit gates for BQSKit. (3) The certified lower bounds (11.1, 12.9) leave a gap of at most a factor ≈ 5. Closing it, either by tightening the bound (deeper spheres, meet-in-the-middle) or by better upper bounds (4-qubit oracles via symmetry reduction), is an open problem that our framework makes precise.

Appendix C Assessment of the reference method

For completeness we list the issues found while reproducing [2], to make the comparison transparent. (1) *Structure.* The gate sets are Clifford, but floating-point comparisons with three different tolerances (10^{-5}, 10^{-4}, 10^{-3}) and a linear database scan are used where exact constant-time keys exist. (2) *Optimality.* Results are compared only with Qiskit. (3) *Equivalence.* Replacements are up to global phase, which is unsound under control, and no error is tracked. (4) *Hardware model.* Discrete angles, all-to-all connectivity and total gate count do not match IBM or ion hardware. (5) *Evaluation.*

Random benchmarks mix (Theorem 7), Tables III–V use one circuit each, and timings compare interpreted MATLAB with compiled transpilers. (6) *Search.* Windows are contiguous and commutation is explored by random swaps without a convergence criterion. (7) *V3.* Forest features and threshold are not reported. (8) *Hardware evidence.* There is one two-qubit circuit, one run per chip and no error bars. (9) *Reproducibility.* Only V2 is released, the loop bounds of `simplereduce` skip the last pair, the database omits the identity, and the database used for Tables III–V is not stated.

References

[1] Nishio, S., Pan, Y., Satoh, T., Amano, H., Van Meter, R.: Extracting success from IBM's 20-qubit machines using error-aware compilation. ACM Journal on Emerging Technologies in Computing Systems **16**(3), 32 (2020)

[2] Rosenhahn, B., Osborne, T.J., Hirche, C.: Optimization driven quantum circuit reduction. New Journal of Physics **27**, 104509 (2025). arXiv:2502.14715

[3] Prasad, A.K., Shende, V.V., Markov, I.L., Hayes, J.P., Patel, K.N.: Data structures and algorithms for simplifying reversible circuits. ACM Journal on Emerging Technologies in Computing Systems **2**(4), 277–293 (2006)

[4] Golubitsky, O., Maslov, D.: A study of optimal 4-bit reversible Toffoli circuits and their synthesis. IEEE Transactions on Computers **61**(9), 1341–1353 (2012)

[5] Nam, Y., Ross, N.J., Su, Y., Childs, A.M., Maslov, D.: Automated optimization of large quantum circuits with continuous parameters. npj Quantum Information **4**, 23 (2018)

[6] Iten, R., Moyard, R., Metger, T., Sutter, D., Woerner, S.: Exact and practical pattern matching for quantum circuit optimization. ACM Transactions on Quantum Computing **3**(1), 4 (2022)

[7] Bravyi, S., Shaydulin, R., Hu, S., Maslov, D.: Clifford circuit optimization with templates and symbolic Pauli gates. Quantum **5**, 580 (2021)

[8] Xu, M., Li, Z., Padon, O., Lin, S., Pointing, J., Hirth, A., Ma, H., Palsberg, J., Aiken, A., Acar, U.A., Jia, Z.: Quartz: superoptimization of quantum circuits. In: Proceedings of the 43rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), pp. 625–640 (2022)

[9] Xu, A., Molavi, A., Pick, L., Tannu, S., Albarghouthi, A.: Synthesizing quantum-circuit optimizers. In: Proceedings of the ACM on Programming Languages (PLDI), vol. 7, pp. 835–859 (2023)

[10] Li, Z., Peng, J., Mei, Y., Lin, S., Wu, Y., Padon, O., Jia, Z.: Quarl: a learning-based quantum circuit optimizer. Proceedings of the ACM on Programming Languages (OOPSLA) **8** (2024)

[11] Javadi-Abhari, A., Treinish, M., Krsulich, K., Wood, C.J., Lishman, J., Gacon, J., Martiel, S., Nation, P.D., Bishop, L.S., Cross, A.W., Johnson, B.R., Gambetta, J.M.: Quantum computing with Qiskit. arXiv:2405.08810 (2024)

[12] Sivarajah, S., Dilkes, S., Cowtan, A., Simmons, W., Edgington, A., Duncan, R.: t|ket⟩: a retargetable compiler for NISQ devices. Quantum Science and Technology **6**(1), 014003 (2021)

[13] Younis, E., Iancu, C.C., Lavrijsen, W., Davis, M., Smith, E.: Berkeley Quantum Synthesis Toolkit (BQSKit) v1. Computer software, Lawrence Berkeley National Laboratory (2021). https://doi.org/10.11578/dc.20210603.2

[14] Hietala, K., Rand, R., Hung, S.-H., Wu, X., Hicks, M.: A verified optimizer for quantum circuits. Proceedings of the ACM on Programming Languages (POPL) **5**, 37–13729 (2021)

[15] Davis, M.G., Smith, E., Tudor, A., Sen, K., Siddiqi, I., Iancu, C.: Towards optimal topology aware quantum circuit synthesis. In: 2020 IEEE International Conference on Quantum Computing and Engineering (QCE), pp. 223–234 (2020)

[16] Smith, E., Davis, M.G., Larson, J., Younis, E., Iancu, C., Lavrijsen, W.: LEAP: Scaling numerical optimization based synthesis using an incremental approach. ACM Transactions on Quantum Computing **4**(1) (2023) https://doi.org/10.1145/3548693

[17] Aaronson, S., Gottesman, D.: Improved simulation of stabilizer circuits. Physical Review A **70**, 052328 (2004)

[18] Selinger, P.: Generators and relations for n-qubit Clifford operators. Logical Methods in Computer Science **11**(2) (2015)

[19] Bravyi, S., Latone, J.A., Maslov, D.: 6-qubit optimal Clifford circuits. npj Quantum Information **8**, 79 (2022)

[20] Duncan, R., Kissinger, A., Perdrix, S., Wetering, J.: Graph-theoretic simplification of quantum circuits with the ZX-calculus. Quantum **4**, 279 (2020)

[21] Giles, B., Selinger, P.: Exact synthesis of multiqubit Clifford+T circuits. Physical Review A **87**, 032332 (2013)

[22] Matsumoto, K., Amano, K.: Representation of quantum circuits with Clifford and $\pi/8$ gates. arXiv:0806.3834 (2008)

[23] Kliuchnikov, V., Maslov, D., Mosca, M.: Fast and efficient exact synthesis of single-qubit unitaries generated by Clifford and T gates. Quantum Information and Computation **13**(7–8), 607–630 (2013)

[24] Ross, N.J., Selinger, P.: Optimal ancilla-free Clifford+T approximation of z-rotations. Quantum Information and Computation **16**(11–12), 901–953 (2016)

[25] Amy, M., Maslov, D., Mosca, M., Roetteler, M.: A meet-in-the-middle algorithm for fast synthesis of depth-optimal quantum circuits. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems **32**(6), 818–830 (2013)

[26] Gosset, D., Kliuchnikov, V., Mosca, M., Russo, V.: An algorithm for the T-count. Quantum Information and Computation **14**(15–16), 1261–1276 (2014)

[27] Tan, B., Cong, J.: Optimality study of existing quantum computing layout synthesis tools. IEEE Transactions on Computers **70**(9), 1363–1373 (2021)

[28] Li, G., Ding, Y., Xie, Y.: Tackling the qubit mapping problem for NISQ-era quantum devices. In: Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pp. 1001–1014 (2019)

[29] Murali, P., Baker, J.M., Javadi-Abhari, A., Chong, F.T., Martonosi, M.: Noise-adaptive compiler mappings for noisy intermediate-scale quantum computers. In: Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pp. 1015–1029 (2019)

[30] Tannu, S.S., Qureshi, M.K.: Not all qubits are created equal: a case for variability-aware policies for NISQ-era quantum computers. In: Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pp. 987–999 (2019)

[31] Cordella, L.P., Foggia, P., Sansone, C., Vento, M.: A (sub)graph isomorphism algorithm for matching large graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence **26**(10), 1367–1372 (2004)

[32] Mølmer, K., Sørensen, A.: Multiparticle entanglement of hot trapped ions. Physical Review Letters **82**, 1835–1838 (1999)

[33] Cartier, P., Foata, D.: Problèmes Combinatoires de Commutation et Réarrangements. Lecture Notes in Mathematics, vol. 85. Springer, Berlin (1969)

[34] Diekert, V., Rozenberg, G. (eds.): The Book of Traces. World Scientific, Singapore (1995)

[35] Vatan, F., Williams, C.: Optimal quantum circuits for general two-qubit gates. Physical Review A **69**, 032315 (2004)

[36] Zhang, J., Vala, J., Sastry, S., Whaley, K.B.: Geometric theory of nonlocal two-qubit operations. Physical Review A **67**, 042313 (2003)

[37] Burgholzer, L., Wille, R.: Advanced equivalence checking for quantum circuits. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems **40**(9), 1810–1824 (2021) https://doi.org/10.1109/TCAD.2020.3032630

[38] Diaconis, P.: Group Representations in Probability and Statistics. IMS Lecture Notes–Monograph Series, vol. 11. Institute of Mathematical Statistics, Hayward, CA (1988)

[39] Li, A., Stein, S., Krishnamoorthy, S., Ang, J.: QASMBench: a low-level quantum benchmark suite for NISQ evaluation and simulation. ACM Transactions on Quantum Computing **4**(2), 10 (2023)

[40] Gromov, M.: Hyperbolic groups. In: Essays in Group Theory. MSRI Publications, vol. 8, pp. 75–263. Springer, New York (1987)

[41] Epstein, D.B.A., Cannon, J.W., Holt, D.F., Levy, S.V.F., Paterson, M.S., Thurston, W.P.: Word Processing in Groups. Jones and Bartlett, Boston (1992)

[42] Knuth, D.E., Bendix, P.B.: Simple word problems in universal algebras. In: Leech, J. (ed.) Computational Problems in Abstract Algebra, pp. 263–297. Pergamon, Oxford (1970)

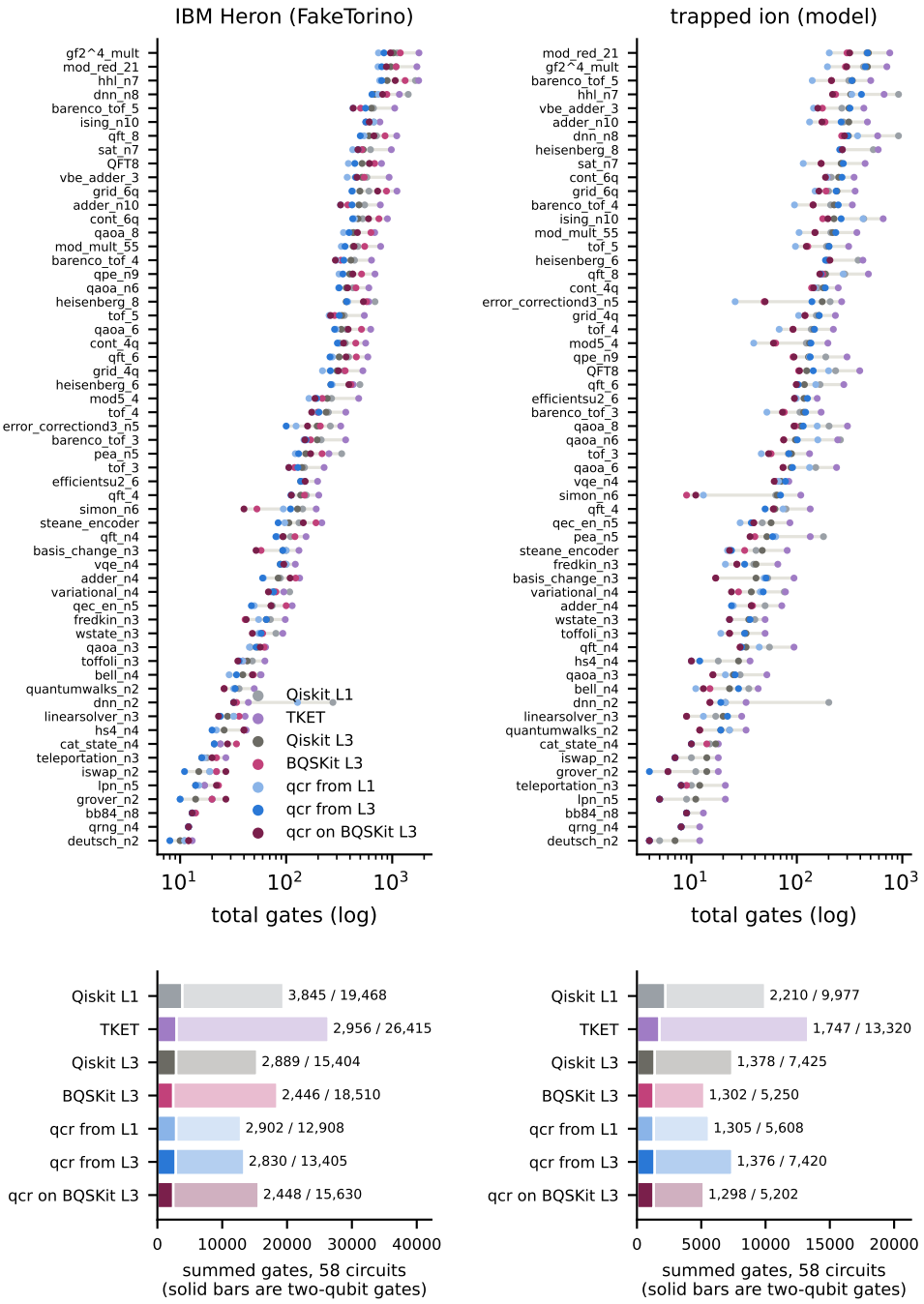
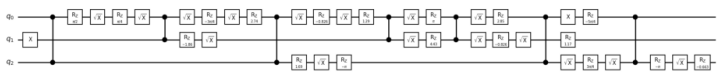
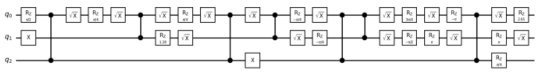


Fig. 7 Hardware suite, from the level-1 compilation to every pipeline's output. The top panel shows total gates per circuit (sorted by the level-3 count), the bottom panel the sums over all circuits, with two-qubit gates as solid bars.

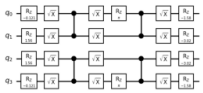
(a) toffoli_n3 after Qiskit level 3 on IBM Heron: 43 gates, 7 CZ



(b) toffoli_n3 after qcr (exact incl. global phase): 36 gates, 7 CZ



(c) hs4_n4 after Qiskit level 3 on IBM Heron: 26 gates, 4 CZ



(d) hs4_n4 after qcr (exact incl. global phase): 20 gates, 4 CZ

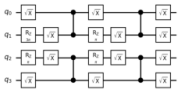


Fig. 8 Two QASMBench circuits after Qiskit level 3 on IBM Heron (a, c) and after `qcr` (b, d). The outputs are equal including global phase.

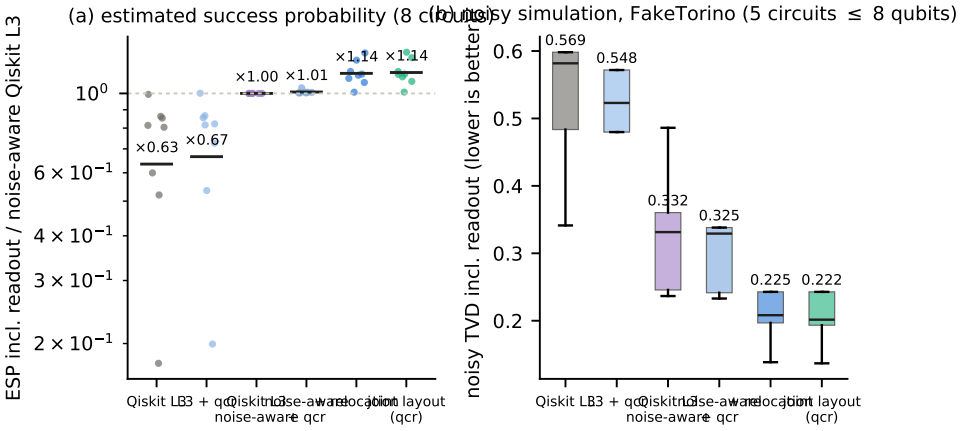


Fig. 9 Layout-aware rewriting on IBM Heron (FakeTorino). (a) ESP including readout relative to Qiskit's noise-aware level 3 (dots are circuits, bars and labels are geometric means). (b) Exact noisy TVD including readout for circuits with at most eight active qubits (labels: means).

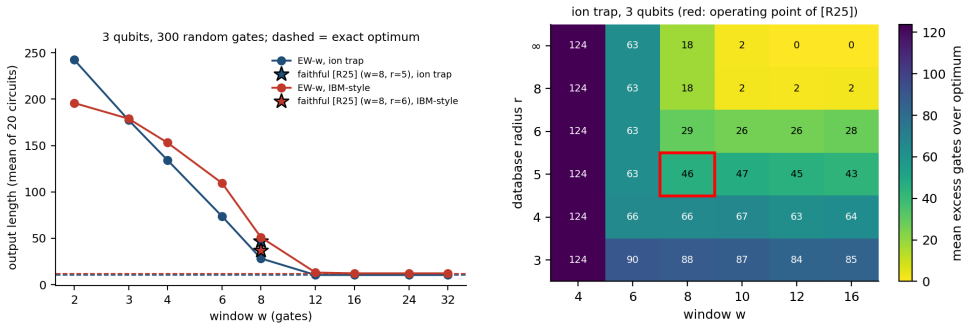


Fig. B1 Left, the fixed-point length of EW-w against w, with the cascade transition near $w^\star$. Right, the mean excess over the optimum by window w and database radius r (ion), with the original operating point $(8, 5)$ marked.